SELECT básico: proyección, filtros con WHERE y operadores de comparación

Los datos de Tienda Latam ya están en las tablas. Ahora viene la parte que vas a usar todos los días: consultarlos. `SELECT` es el comando más ejecutado en SQL, y la diferencia entre una consulta bien escrita y una mal escrita no es solo estética. Una consulta que trae más datos de los que necesitas es más lenta, más difícil de leer y más propensa a errores de interpretación.

La estructura base: SELECT y FROM

Todo empieza con dos palabras que siempre van juntas: `SELECT` y `FROM`. La primera dice qué columnas quieres ver; la segunda dice de qué tabla.

```sql
SELECT * FROM clientes LIMIT 10;
```

El asterisco (`*`) trae todas las columnas de la tabla. `LIMIT` restringe cuántas filas devuelve la consulta. Es una buena práctica usarlo mientras explores los datos, para no traer miles de filas de golpe.

Pero en la práctica, traer todas las columnas con `*` rara vez es lo que necesitas. La mayoría de las veces quieres columnas específicas:

```sql
SELECT nombre, correo, activo

FROM clientes

LIMIT 10;
```

Aquí le estás diciendo exactamente qué quieres ver. Menos datos, respuesta más rápida, resultado más fácil de leer.

WHERE: filtrar para quedarte solo con lo que importa

`WHERE` va después del `FROM` y le pone condiciones al resultado. Solo devuelve las filas que cumplen con lo que pides.

Condición simple

```sql
SELECT nombre, correo

FROM clientes

WHERE activo = TRUE;
```

Solo devuelve los clientes que tienen `activo = TRUE`. Los que tienen `FALSE` no aparecen.

Comparadores numéricos

```sql
SELECT nombre, precio

FROM productos

WHERE precio > 100;
```

Puedes usar `>`, `<`, `>=`, `<=` y `=` para comparar números. En este caso, solo devuelve productos con precio mayor a cien. Un precio de exactamente cien no entra; tendría que ser `>= 100` para incluirlo.

Rango con BETWEEN

```sql
SELECT nombre, precio

FROM productos

WHERE precio BETWEEN 50 AND 200;
```

`BETWEEN` incluye los extremos: devuelve todo lo que esté entre 50 y 200, incluyendo esos dos valores.

Lista con IN

Cuando quieres filtrar por varios valores específicos de un mismo campo, en lugar de escribir un `OR` para cada uno usas `IN` con una lista:

```sql
SELECT nombre, codigo

FROM paises

WHERE codigo IN ('CHL', 'ARG', 'COL');
```

Solo devuelve los países cuyo código coincide exactamente con alguno de los tres valores de la lista. Funciona con cualquier cantidad de elementos.

Texto parcial con LIKE

Cuando sabes solo una parte del texto que buscas, usas `LIKE` con el símbolo `%`, que representa cualquier secuencia de caracteres.

Buscar lo que empieza con algo:

```sql
SELECT nombre, correo

FROM clientes

WHERE nombre LIKE 'Mar%';
```

Devuelve todos los clientes cuyo nombre empieza con "Mar": María, Marcelo, Martina, etc.

Buscar lo que termina con algo:

```sql
SELECT nombre, correo

FROM clientes

WHERE correo LIKE '%@gmail.com';
```

Devuelve todos los clientes cuyo correo termina en `@gmail.com`, sin importar lo que haya antes.

El `%` puede ir al inicio, al final o en ambos lados dependiendo de qué parte del texto conoces.

Valores nulos con IS NULL / IS NOT NULL

`NULL` no es un valor vacío ni un cero: es la ausencia de dato. Por eso no se puede comparar con `=`; tiene su propia sintaxis.

```sql
-- Clientes sin teléfono registrado:

SELECT nombre, telefono

FROM clientes

WHERE telefono IS NULL;


-- Clientes que sí tienen teléfono:

SELECT nombre, telefono

FROM clientes

WHERE telefono IS NOT NULL;
```

Combinar condiciones: AND, OR y NOT

Las condiciones del `WHERE` se pueden combinar con lógica.

AND — se tienen que cumplir todas las condiciones:

```sql
SELECT nombre, precio, stock

FROM productos

WHERE precio > 50 AND stock > 0;
```

Solo devuelve productos que cumplan las dos condiciones a la vez. Si el precio es mayor a 50 pero el stock es cero, no aparece.

OR — basta con que se cumpla una:

```sql
SELECT nombre, codigo

FROM paises

WHERE codigo = 'CHL' OR codigo = 'ARG';
```

Devuelve cualquier fila que cumpla al menos una de las dos condiciones. Para este caso con pocos valores, `IN` sería más limpio; pero `OR` es útil cuando las condiciones son sobre campos distintos.

NOT — invierte la condición:

```sql
SELECT nombre, activo

FROM clientes

WHERE NOT activo = FALSE;
```

`NOT` da vuelta el resultado de la condición que le sigue. En este caso, devuelve los clientes cuyo `activo` no es `FALSE`, es decir, los que sí están activos. Es equivalente a `WHERE activo = TRUE`, pero `NOT` es útil cuando la condición que quieres excluir es más fácil de expresar que la que quieres incluir.

ORDER BY: ordenar los resultados

Por defecto, el motor devuelve las filas en el orden en que están guardadas, que puede ser cualquiera. Si necesitas un orden específico, usas `ORDER BY`.

```sql
-- De mayor a menor precio:
SELECT nombre, precio
FROM productos
ORDER BY precio DESC;

-- De menor a mayor precio:
SELECT nombre, precio
FROM productos
ORDER BY precio ASC;
```

`DESC` es descendente (de mayor a menor). `ASC` es ascendente (de menor a mayor) y es el valor por defecto si no escribes nada.

AS: ponerle nombre a las columnas del resultado

Los nombres de las columnas en el resultado pueden cambiarse con AS. Esto no modifica la tabla ni el campo; solo cambia cómo se llama la columna en el resultado de esa consulta.

```SQL
SELECT
    nombre        AS cliente,
    correo        AS correo_electronico,
    activo        AS vigente
FROM clientes;
```

El resultado muestra "cliente", "correo_electronico" y "vigente" en lugar de los nombres originales. Útil cuando el resultado va a mostrarse en un reporte o cuando los nombres de los campos no son suficientemente descriptivos por sí solos.

Referencia rápida de filtros

Qué quieres filtrar	Qué usar
Un valor exacto	`WHERE campo = valor`
Un rango de números	`WHERE campo BETWEEN x AND y`
Varios valores posibles	`WHERE campo IN (a, b, c)`
Texto que empieza, termina o contiene algo	`WHERE campo LIKE 'patrón%'`

Campos sin dato	`WHERE campo IS NULL`
Campos con dato	`WHERE campo IS NOT NULL`
Dos condiciones obligatorias	`WHERE condicion1 AND condicion2`
Al menos una condición	`WHERE condicion1 OR condicion2`
Excluir una condición	`WHERE NOT condicion`

Hasta ahora todas las consultas trabajan con una sola tabla. Pero la base de datos de Tienda Latam tiene nueve tablas relacionadas entre sí. Para sacar información cruzada entre ellas, como qué productos compró un cliente o qué empleado atendió cada pedido, necesitas aprender a combinar tablas en una sola consulta.